R Programming Essentials

A Fast Practical Guide

Version 1.0

For beginners who need to become productive in R for data analysis quickly.
Base R fundamentals, then dplyr, tidyr, readr, stringr, lubridate and ggplot2.
Every code listing was executed; every output block is the real output.

Requirements: R 4.1 or later. Output generated with R 4.3.3, dplyr 1.1.4, tidyr 1.3.1, readr 2.1.5, stringr 1.5.1, lubridate 1.9.3, ggplot2 3.4.4, readxl 1.4.3.

Contents

1. Setup and First Steps 3

2. Data Types and Variables 5

3. Data Structures 8

4. Control Flow 13

5. Functions 15

6. Importing and Exporting Data 17

7. Data Wrangling 19

8. Data Visualization 23

9. Statistics Essentials 26

10. Mini Projects 29

Appendix: Cheat Sheet and Next Steps 32

Answers to Exercises 34

How to use this guide

Read the chapters in order; each one assumes only the chapters before it. Type the examples rather than pasting them, and change a value to see what happens. Every chapter lists the packages it needs, ends with graded exercises, and the answers are collected at the end of the document.

Element	Meaning
Grey code box	R code. A small label gives the kind: **BASE R** needs nothing extra; **PACKAGE CODE** names the packages used; *(not run)* marks code shown but not executed
Blue output box	The exact output of the code above it, produced by running it in R 4.3.3
Common Mistake	A frequent error and how to avoid it
Best Practice	A habit that saves time later
Version Note	Behavior that depends on the R version or on a package version
Exercises	Three to five tasks per chapter, from easy to hard; answers at the end

Install the packages once with `install.packages(c("dplyr", "tidyr", "readr", "readxl", "stringr", "lubridate", "ggplot2", "tibble"))`. All data used are built into R or shipped with these packages, so no downloads are needed.

1. Setup and First Steps

Packages required: base R only, plus dplyr for one demonstration of package loading.

R is a free language for statistics and data analysis. RStudio is a separate program (an IDE) that makes R easier to use. Install R first from CRAN (cran.r-project.org), then install RStudio Desktop from Posit. Every code block in this guide carries a small label: **BASE R** means the code needs nothing beyond a standard R installation; **PACKAGE CODE** lists the add-on packages it uses. Grey boxes are code; blue boxes are the output R produces.

Console, scripts and the first calculations

The console is where R executes commands immediately; it is ideal for experiments. A script (a plain text file ending in `.R`) stores commands so you can rerun, edit and share them. Work in scripts and send lines to the console with Ctrl+Enter (Cmd+Enter on macOS). Everything you do in analysis should be reproducible from a script.

```
BASE R
2 + 3 * 4            # arithmetic follows the usual precedence rules
10 %/% 3             # integer division
10 %% 3              # remainder (modulo)
2^10                 # power
sqrt(16)             # a function call: name(argument)
round(3.14159, digits = 2)   # named argument
c(4, 8, 15, 16)      # c() combines values into a vector
1:5                  # shortcut for a sequence of integers
```

```
[1] 14
[1] 3
[1] 1
[1] 1024
[1] 4
[1] 3.14
[1]  4  8 15 16
[1] 1 2 3 4 5
```

Anything after `#` is a comment and is ignored by R. Comment the reason for a step, not the obvious mechanics.

Projects and the working directory

The working directory is the folder R reads files from and writes files to. Check it with `getwd()`. Use an RStudio Project (File > New Project) for each analysis: opening the project sets the working directory to the project folder, so relative paths such as `"data/sales.csv"` keep working on any computer.

```
BASE R  (not run)
getwd()                      # print the current working directory
list.files()                 # list files in it
file.path("data", "sales.csv")  # build a path portably
```

> **Best Practice.** One folder per analysis, opened as an RStudio Project. Avoid `setwd("C:/Users/me/...")` inside scripts: absolute paths break as soon as the script moves to another machine. In RStudio, also set Tools > Global Options > General > "Save workspace to .RData on exit" to Never, so every session starts clean.

Installing and loading packages

A package is a bundle of functions and data. Install it once per computer with `install.packages()`; load it in every new session with `library()`. Note the quotes: the name is a string when installing, but a bare name is accepted by `library()`.

```
PACKAGE CODE: dplyr  (not run)
install.packages("dplyr")     # once per computer (needs an internet connection)
install.packages("tidyverse") # installs the core tidyverse packages together
library(dplyr)                # every session: attach the package
dplyr::filter(mtcars, cyl == 4)   # pkg::fun calls a function without attaching
```

Loading a package can mask a function of the same name from another package. R reports this when the package is attached; the most recently loaded package wins, and `pkg::fun` removes any ambiguity.

```
PACKAGE CODE: dplyr
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

Common Mistake. Writing `install.packages(dplyr)` without quotes fails with "object 'dplyr' not found", and putting `install.packages()` at the top of a shared script re-installs the package on every run. Install interactively, load in the script.

Getting help

R documentation is built in and consistent. Learn to read a help page: Usage shows the arguments and defaults, Arguments explains them, Value says what comes back, and Examples can be run as written.

Command	What it does
`?mean` or `help("mean")`	Open the help page of a function
`??regression` or `help.search("regression")`	Search all installed help pages for a word
`example(mean)`	Run the examples from a help page
`args(sd)`	Print the argument list of a function
`apropos("read")`	List function names containing a word
`vignette()` and `browseVignettes("dplyr")`	Long-form tutorials shipped with packages

```
BASE R
args(sd)
R.version.string
```

```
function (x, na.rm = FALSE)
NULL
[1] "R version 4.3.3 (2024-02-29)"
```

Structure of a script

A readable script follows the same order every time: description, packages, settings and input, functions, analysis, output. RStudio folds sections that end in four dashes. Useful shortcuts: Alt+- inserts `<-`, Ctrl+Shift+M inserts a pipe, Ctrl+Shift+C comments or uncomments the selected lines.

```
PACKAGE CODE: dplyr  (not run)
# Purpose: summarise fuel economy by cylinder count
# Author:  <name>        Date: <date>

# 1. Packages ----
library(dplyr)

# 2. Input ----
cars <- mtcars

# 3. Analysis ----
result <- cars |> group_by(cyl) |> summarise(mean_mpg = mean(mpg))

# 4. Output ----
print(result)
```

Exercises

1.1 [Easy] Compute (15 + 25) / 4 and the remainder when 47 is divided by 6.

1.2 [Easy] Print the argument list of the function `round` and read off the default number of digits.

1.3 [Medium] Create the vector of the squares of the integers 1 to 10 and compute its mean.

1.4 [Medium] Create an object `a`, list the objects in your session, remove everything, and list again.

2. Data Types and Variables

Packages required: base R only.

Assignment and names

Assign a value to a name with `<-`. The `=` sign also works at the top level, but the tidyverse style guide reserves `=` for function arguments, which keeps the two roles visually distinct. Names are case-sensitive, may contain letters, digits, `.` and `_`, and must not start with a digit. Use lowercase snake_case.

```
BASE R
price <- 19.99          # preferred assignment
quantity = 3            # works, but avoid for assignment
total <- price * quantity
total
```

```
[1] 59.97
```

The basic types

Every value has a type that decides which operations are allowed. `class()` reports what a value is for R users; `typeof()` reports the internal storage type.

Type	Example	Notes
numeric (double)	`3.5`, `1e3`	Default for any number typed without a suffix
integer	`4L`	Suffix `L`; whole numbers only
character	`"text"`	Single or double quotes
logical	`TRUE`, `FALSE`	Also written `T` and `F`; never rely on those
factor	`factor("a")`	Categorical data with fixed levels
Date	`as.Date("2024-03-15")`	Stored as days since 1970-01-01
NULL	`NULL`	The empty object, length zero
NA	`NA`	A missing value of any type

```
BASE R
x <- 3.5; n <- 4L; s <- "text"; flag <- TRUE
c(class(x), class(n), class(s), class(flag))
typeof(x)
typeof(n)
is.numeric(n)         # integers count as numeric
```

```
[1] "numeric"   "integer"   "character" "logical"
[1] "double"
[1] "integer"
[1] TRUE
```

Floating-point numbers are stored with limited precision, so never compare them with `==`. Use `all.equal()` wrapped in `isTRUE()`.

```
BASE R
0.1 + 0.2 == 0.3
isTRUE(all.equal(0.1 + 0.2, 0.3))
5 / 2                # ordinary division always returns a double
5L %/% 2L            # integer division of integers returns an integer
```

```
[1] FALSE
[1] TRUE
[1] 2.5
[1] 2
```

Common Mistake. Comparing decimals with ==. The first line above prints FALSE although the arithmetic looks exact. Tolerance-based comparison is the correct tool.

Character, logical and factor values

Strings are created with quotes and manipulated with functions such as `paste()`, `nchar()` and `toupper()`. Logical values come from comparisons and combine with & (and), | (or) and ! (not). Because `TRUE` counts as 1 and `FALSE` as 0, `sum()` counts matches and `mean()` gives a proportion.

```
BASE R
first <- "Ada"; last <- "Lovelace"
paste(first, last)             # joins with a space
paste0(first, "_", last)       # joins with no separator
nchar(last)
toupper(last)
ages <- c(23, 35, 41, 19)
ages > 20
sum(ages > 20)                 # how many are over 20
mean(ages > 20)                # proportion over 20
```

```
[1] "Ada Lovelace"
[1] "Ada_Lovelace"
[1] 8
[1] "LOVELACE"
[1]  TRUE  TRUE  TRUE FALSE
[1] 3
[1] 0.75
```

A factor stores categories as integer codes with a fixed set of labelled levels. Use it for categorical variables so that the order of categories is explicit.

```
BASE R
size <- factor(c("low", "high", "medium", "low"),
               levels = c("low", "medium", "high"))
size
levels(size)
as.integer(size)              # the underlying codes
table(size)                   # counts per level
```

```
[1] low    high   medium low
Levels: low medium high
[1] "low"    "medium" "high"
[1] 1 3 2 1
size
   low medium   high
     2      1      1
```

Version Note. Before R 4.0.0, `data.frame()` and `read.csv()` converted text columns to factors by default (`stringsAsFactors = TRUE`). Since R 4.0.0 the default is FALSE. Code and tutorials written for older versions may assume the old behavior.

Dates, NULL and NA

Dates need an explicit conversion from text. The default format is year-month-day. Arithmetic works in days.

```
BASE R
d <- as.Date("2024-03-15")
d + 30
format(d, "%d/%m/%Y")          # format codes: %d day, %m month, %Y year
as.numeric(d)                  # days since 1970-01-01
as.Date("2024-12-25") - d      # difference of two dates
```

```
[1] "2024-04-14"
[1] "15/03/2024"
[1] 19797
Time difference of 285 days
```

NA marks a missing value and is contagious: most operations on NA return NA. Test for it with is.na(), never with == NA. NULL means no object at all; it has length zero and disappears when combined into a vector.

```
BASE R
v <- c(4, NA, 8)
mean(v)
mean(v, na.rm = TRUE)          # remove NA before computing
v == NA                        # useless: always NA
is.na(v)
length(NULL)
c(1, NULL, 3)
```

```
[1] NA
[1] 6
[1] NA NA NA
[1] FALSE  TRUE FALSE
[1] 0
[1] 1 3
```

Type checking and coercion

Functions named is.*() test a type and as.*() convert to it. A failed conversion returns NA with a warning. When values of different types are combined in one vector, R coerces silently to the most flexible type in the order logical, integer, numeric, character.

```
BASE R
as.numeric("12.5")
as.numeric("abc")              # NA with a warning
as.integer(3.9)                # truncates toward zero
as.logical(c("TRUE", "true", "T", "yes"))
c(1, "a", TRUE)               # everything becomes character
TRUE + TRUE                    # logical becomes numeric
```

```
[1] 12.5
[1] NA
Warning message:
NAs introduced by coercion
[1] 3
[1] TRUE TRUE TRUE   NA
[1] "1"    "a"    "TRUE"
[1] 2
```

Common Mistake. Converting a factor of numbers with as.numeric() returns the level codes, not the values. Convert via character first: as.numeric(as.character(f)).

```
BASE R
f <- factor(c("10", "20", "10"))
as.numeric(f)                  # wrong: level codes
as.numeric(as.character(f))    # correct
```

```
[1] 1 2 1
[1] 10 20 10
```

Vectorization basics

Most R operations work on whole vectors at once, element by element. This removes most loops and is the main reason R code is short. When two vectors differ in length, the shorter one is recycled; R warns when the longer length is not a multiple of the shorter.

```
BASE R
heights <- c(1.62, 1.75, 1.80)
weights <- c(58, 72, 85)
weights / heights^2            # element-wise arithmetic
sqrt(c(1, 4, 9))
nchar(c("a", "bb", "ccc"))
1:6 + 1:2                      # recycling, lengths compatible
1:5 + 1:2                      # recycling with a warning
```

```
[1] 22.10029 23.51020 26.23457
[1] 1 2 3
[1] 1 2 3
[1] 2 4 4 6 6 8
[1] 2 4 4 6 6
Warning message:
In 1:5 + 1:2 :
  longer object length is not a multiple of shorter object length
```

Exercises

2.1 [Easy] Create the integer 7 and the double 7. Show the class of each.

2.2 [Easy] Write a correct test of whether `0.1 + 0.2` equals `0.3`.

2.3 [Medium] Convert `c("10", "20", "x")` to numbers without printing a warning, then count the missing values.

2.4 [Medium] Make an ordered factor from `c("S", "L", "M", "S", "L")` with order S < M < L and count each level.

2.5 [Hard] Compute the number of days from 2024-01-01 to 2024-12-25, and the date 100 days after 2024-01-01.

3. Data Structures

Packages required: base R, plus tibble (part of the tidyverse) for the tibble section.

R has five structures you will use constantly. The first three hold one type only; the list and the data frame can mix types.

Structure	Dimensions	Types	Typical use
vector	1	one type	A column of values, results of calculations
matrix	2	one type	Numeric tables, linear algebra
array	3 or more	one type	Stacks of matrices
list	1	mixed	Collections of unrelated objects, function results
data frame	2	mixed per column	A dataset: rows are observations, columns are variables

Vectors and indexing

Create vectors with `c()`, `:`, `seq()` and `rep()`. Indexing uses square brackets and starts at 1, not 0. You can index with positive positions, negative positions (exclusion), logical vectors, or names.

```
BASE R
x <- c(10, 20, 30, 40, 50)
x[2]                  # second element
x[c(1, 5)]            # several positions
x[-1]                 # everything except the first
x[x > 25]             # logical index: keep elements where TRUE
which(x > 25)         # positions where the condition holds
seq(0, 1, by = 0.25)
rep(c("a", "b"), times = 2)
scores <- c(ana = 90, ben = 72, cy = 85)   # a named vector
scores["ben"]
```

```
[1] 20
[1] 10 50
[1] 20 30 40 50
[1] 30 40 50
[1] 3 4 5
[1] 0.00 0.25 0.50 0.75 1.00
[1] "a" "b" "a" "b"
ben
 72
```

Assigning through an index modifies elements in place, and indexing beyond the end extends the vector with `NA`. Useful companions are `length()`, `rev()`, `sort()`, `order()`, `head()`, `tail()`, `unique()`, `%in%` and `sum()`.

```
BASE R
x[2] <- 99
x[7] <- 1             # position 6 is filled with NA
x
c(2, 9) %in% x        # membership test
sort(c(3, 1, 2), decreasing = TRUE)
```

```
[1] 10 99 30 40 50 NA  1
[1] FALSE FALSE
[1] 3 2 1
```

Lists

A list holds elements of any type, including other lists. Single brackets return a smaller list; double brackets, or `$` for named elements, extract the element itself.

```
BASE R
person <- list(name = "Ada", age = 36, langs = c("R", "Python"))
person$name
person[["langs"]]
person["age"]         # still a list of length 1
class(person[["age"]])
person$email <- "ada@example.org"   # add an element
names(person)
str(person)           # compact view of any object
```

```
[1] "Ada"
[1] "R"      "Python"
$age
[1] 36

[1] "numeric"
[1] "name"  "age"   "langs" "email"
List of 4
 $ name : chr "Ada"
 $ age  : num 36
 $ langs: chr [1:2] "R" "Python"
 $ email: chr "ada@example.org"
```

Matrices and arrays

A matrix is a vector with a dimension attribute. `matrix()` fills column by column unless `byrow = TRUE`. Index with `[row, column]`; leave a position empty to take all of it.

```
BASE R
m <- matrix(1:6, nrow = 2, ncol = 3)
m
m[2, 3]               # row 2, column 3
m[, 2]                # whole second column
dim(m)
t(m)                  # transpose
m * 10                # element-wise arithmetic
m %*% t(m)            # matrix multiplication
rownames(m) <- c("r1", "r2")
cbind(m, total = rowSums(m))
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
[1] 6
[1] 3 4
[1] 2 3
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
     [,1] [,2] [,3]
[1,]   10   30   50
[2,]   20   40   60
     [,1] [,2]
[1,]   35   44
[2,]   44   56
          total
r1 1 3 5     9
r2 2 4 6    12
```

An array generalises a matrix to more dimensions and is indexed with one position per dimension.

```
BASE R
a <- array(1:24, dim = c(2, 3, 4))
dim(a)
a[2, 3, 4]
a[, , 1]              # first 2 x 3 layer
```

```
[1] 2 3 4
[1] 24
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Data frames

A data frame is a list of equal-length columns, so each column has its own type. It is the standard container for datasets. Extract a column with `$` or `[[`, and use `[rows, columns]` for sub-tables. `str()`, `head()` and `summary()` give a quick inspection.

```
BASE R
df <- data.frame(
  id    = 1:4,
  group = c("a", "b", "a", "b"),
  score = c(5.1, 6.3, 4.8, 7.0)
)
df
str(df)
df$score
df[df$score > 5, ]            # rows where the condition holds
df[, c("id", "score")]        # selected columns
df$pass <- df$score >= 5      # new column
df[order(-df$score), ]        # sort by score, descending
nrow(df); ncol(df)
```

```
  id group score
1  1     a   5.1
2  2     b   6.3
3  3     a   4.8
4  4     b   7.0
'data.frame':   4 obs. of  3 variables:
 $ id   : int  1 2 3 4
 $ group: chr  "a" "b" "a" "b"
 $ score: num  5.1 6.3 4.8 7
[1] 5.1 6.3 4.8 7.0
  id group score
1  1     a   5.1
2  2     b   6.3
4  4     b   7.0
  id score
1  1   5.1
2  2   6.3
3  3   4.8
4  4   7.0
  id group score  pass
4  4     b   7.0  TRUE
2  2     b   6.3  TRUE
1  1     a   5.1  TRUE
3  3     a   4.8 FALSE
[1] 4
[1] 4
```

Combine data frames with `rbind()` (same columns, more rows) and `cbind()` (same rows, more columns). Built-in datasets such as `mtcars` and `iris` are data frames.

```
BASE R
rbind(df[1:2, 1:3], data.frame(id = 5L, group = "c", score = 5.5))
head(mtcars, 3)
```

```
  id group score
1  1     a   5.1
2  2     b   6.3
3  5     c   5.5
                mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710     22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
```

Tibbles

A tibble is a modern data frame from the tibble package. It prints only the first rows and the columns that fit on screen, shows column types, never converts text to factors, and does not partially match column names. Subsetting with `[` always returns a tibble.

```
PACKAGE CODE: tibble
library(tibble)
tb <- as_tibble(df)
tb
tb[, "score"]                 # stays a tibble
df[, "score"]                 # base data frame: drops to a vector
```

```
# A tibble: 4 × 4
     id group score pass
  <int> <chr> <dbl> <lgl>
1     1 a       5.1 TRUE
2     2 b       6.3 TRUE
3     3 a       4.8 FALSE
4     4 b       7   TRUE
# A tibble: 4 × 1
  score
  <dbl>
1   5.1
2   6.3
3   4.8
4   7
[1] 5.1 6.3 4.8 7.0
```

Common indexing mistakes

Common Mistake. Four errors cause most indexing bugs: (1) assuming indexing starts at 0; (2) writing `x[x = 3]` or `if (x = 3)` instead of `==`; (3) forgetting that `df[, 1]` returns a vector, so use `df[, 1, drop = FALSE]` to keep a data frame; (4) using `[` where `[[` is needed, which returns a list instead of the element.

```
BASE R
x <- c(5, NA, 8)
x[x > 6]                      # NA in the condition yields NA in the result
x[which(x > 6)]               # which() drops NA positions
x[x > 6 & !is.na(x)]          # explicit alternative
x[0]                          # zero index returns an empty vector
```

```
[1] NA  8
[1] 8
[1] 8
numeric(0)
```

Best Practice. Prefer names over positions (`df[["score"]]`, `df[, "score"]`) so code survives a change in column order, and inspect every new object with `str()` before using it.

Exercises

3.1 [Easy] From `x <- c(4, 8, 15, 16, 23, 42)` extract the last element and every element greater than 10.

3.2 [Easy] Create a list with a name, an age and a vector of three hobbies, then extract the second hobby.

3.3 [Medium] Build a 3 x 3 matrix containing 1 to 9 filled by rows and compute its row sums.

3.4 [Medium] In `mtcars`, select the cars with 6 cylinders and keep only the columns `mpg` and `hp`.

3.5 [Hard] Add a column `kpl` (kilometres per litre, `mpg * 0.425144`) to the first ten rows of `mtcars` and show the three cars with the highest `kpl`.

4. Control Flow

Packages required: base R only.

Control flow decides which code runs and how often. In R, prefer vectorized operations (Chapter 2) and use explicit loops only when each step depends on the previous one or when you must call something once per item.

Conditions: if, else, ifelse and switch

`if` tests a single TRUE or FALSE. Use `else if` for further branches. The braces are required for multi-line bodies, and `else` must start on the same line as the closing brace of the `if` block.

```
BASE R
temperature <- 31
if (temperature > 30) {
  label <- "hot"
} else if (temperature > 15) {
  label <- "mild"
} else {
  label <- "cold"
}
label
```

```
[1] "hot"
```

`ifelse(test, yes, no)` is the vectorized version: it works on a whole vector and returns a vector of the same length. `switch()` picks one of several results by name, and its unnamed last argument is the default.

```
BASE R
temps <- c(31, 12, 22, NA)
ifelse(temps > 20, "warm", "cool")      # NA stays NA
unit <- "km"
switch(unit,
       km = 1000,
       m  = 1,
       cm = 0.01,
       NA)                              # default for unknown units
```

```
[1] "warm" "cool" "warm" NA
[1] 1000
```

Combine conditions with `&&` and `||` inside `if`: they accept single values only and stop evaluating as soon as the answer is known. Use `&` and `|` for vectors.

> **Version Note.** Since R 4.2.0 an `if` or `while` condition of length greater than one is an error (it was a warning before). Since R 4.3.0 the same applies to `&&` and `||`. Code that relied on the old behavior must use `any()`, `all()` or `isTRUE()`.

```
BASE R
tryCatch(if (c(TRUE, FALSE)) "yes", error = function(e) conditionMessage(e))
x <- c(3, 8)
if (any(x > 5)) "at least one above 5"
```

```
[1] "the condition has length > 1"
[1] "at least one above 5"
```

for loops

A `for` loop runs its body once per element. Loop over `seq_along(x)` rather than `1:length(x)`: when `x` is empty, `1:length(x)` is `1:0`, which iterates over 1 and 0. Preallocate the result with `numeric(n)` or `vector("list", n)`; growing a vector inside the loop is slow.

```
BASE R
squares <- numeric(5)                    # preallocate
for (i in seq_along(squares)) {
  squares[i] <- i^2
}
squares
for (name in c("ana", "ben")) {
  cat("Hello,", name, "\n")              # cat prints without [1] and quotes
}
```

```
[1]  1  4  9 16 25
Hello, ana
Hello, ben
```

Common Mistake. `for (i in 1:length(x))` on an empty vector, and growing results with `result <- c(result, value)` inside a loop. Use `seq_along()` and preallocation.

while, repeat, break and next

`while` repeats as long as its condition is TRUE. `repeat` loops forever until `break`. Inside any loop, `break` leaves the loop and `next` skips to the next iteration. Make sure a `while` condition can eventually become FALSE.

```
BASE R
n <- 1
while (n < 100) {                        # double until the value reaches 100
  n <- n * 2
}
n
count <- 0
repeat {
  count <- count + 1
  if (count %% 2 == 0) next              # skip even numbers
  if (count > 7) break                   # stop the loop
  cat("odd:", count, "\n")
}
```

```
[1] 128
odd: 1
odd: 3
odd: 5
odd: 7
```

Vectorized alternatives

Most loops that compute something per element can be written as one expression. This is shorter, clearer and usually faster.

Loop pattern	Vectorized form
Add up the positive values	`sum(x[x > 0])`
Running total	`cumsum(x)`
Label each element by a rule	`ifelse(x > 0, "pos", "neg")`
Count elements meeting a rule	`sum(x > 10)`
Apply a function to each element	`sapply(x, f)` or `lapply(x, f)` (Chapter 5)

```
BASE R
x <- c(-2, 5, -1, 8, 3)
total <- 0
for (v in x) if (v > 0) total <- total + v   # loop version
total
sum(x[x > 0])                                # vectorized version
cumsum(x)
```

```
[1] 16
[1] 16
[1] -2  3  2 10 13
```

Best Practice. Write the loop only when you cannot express the step as a vector operation. When you do loop, keep the body small, name the loop variable meaningfully and test the code on a short input first.

Exercises

4.1 [Easy] Write an `if/else` that prints "even" or "odd" for the number 17.

4.2 [Easy] Use `ifelse()` to turn `c(5, 12, 20, 3)` into "high" (above 10) and "low".

4.3 [Medium] Use a `for` loop to compute 10 factorial and check it with `factorial(10)`.

4.4 [Medium] Use `while` to find the first power of 3 that exceeds 1000.

4.5 [Hard] Replace a loop by a vectorized expression: count the numbers between 1 and 100 that are divisible by 3 or by 5.

5. Functions

Packages required: base R only.

A function packages a computation under a name so that you write it once and reuse it. If you copy and paste the same code three times, write a function instead.

Defining functions

A function has arguments, a body and a return value. The value of the last evaluated expression is returned automatically; use `return()` only to exit early. Arguments can have defaults, and callers can pass them by position or by name.

```
BASE R
bmi <- function(weight_kg, height_m, digits = 1) {
  value <- weight_kg / height_m^2
  round(value, digits)                  # last expression is the result
}
bmi(70, 1.75)
bmi(height_m = 1.75, weight_kg = 70, digits = 2)   # named arguments, any order
bmi(c(60, 80), c(1.6, 1.8))            # vectorized because the body is
```

```
[1] 22.9
[1] 22.86
[1] 23.4 24.7
```

A function returns one object. To return several values, return a list.

```
BASE R
describe <- function(x) {
  list(mean = mean(x), sd = sd(x), n = length(x))
}
res <- describe(c(4, 8, 15, 16, 23, 42))
res$mean
res$n
```

```
[1] 18
[1] 6
```

The special argument `...` forwards extra arguments to another function, which is how wrappers stay flexible. Use `stopifnot()` to check inputs early.

```
BASE R
safe_mean <- function(x, ...) {
  stopifnot(is.numeric(x))              # fails with a clear message otherwise
  mean(x, ...)
}
safe_mean(c(1, NA, 3), na.rm = TRUE)    # na.rm travels through ...
```

```
[1] 2
```

Scope

Variables created inside a function are local and disappear when it returns. A function can read variables from the enclosing environment, but assigning to a name inside a function creates a new local variable and leaves the global one untouched. Pass data in through arguments and return results explicitly.

```
BASE R
counter <- 10
change <- function() {
  counter <- 99                          # local copy only
  counter
}
change()
counter                                  # the global value is unchanged
```

```
[1] 99
[1] 10
```

Common Mistake. Using the super-assignment `<<-` to modify global state from inside a function. It makes code hard to follow and test. Return the new value instead and assign it where the function was called.

Anonymous functions and the apply family

A function without a name is useful as a short argument to another function. `function(x) x^2` works in every R version; the shorthand `\(x) x^2` requires R 4.1.0 or later.

The apply family replaces many loops. `lapply()` always returns a list. `sapply()` simplifies the result to a vector or matrix when it can, which makes its return type unpredictable. `vapply()` makes you declare the expected type and length with `FUN.VALUE`, so it is the safe choice in scripts and packages. `Map()` calls a function over several vectors in parallel.

```
BASE R
nums <- c(a = 1, b = 4, c = 9)
lapply(nums, sqrt)
sapply(nums, sqrt)                       # simplified to a named vector
vapply(nums, \(v) v * 2, numeric(1))     # declare: one number per element
sapply(list(), length)                   # empty input: returns an empty list
vapply(list(), length, integer(1))       # vapply keeps the type
Map(function(x, y) x + y, 1:3, 4:6)
```

```
$a
[1] 1

$b
[1] 2

$c
[1] 3

a b c
1 2 3
 a  b  c
 2  8 18
list()
integer(0)
[[1]]
[1] 5

[[2]]
[1] 7

[[3]]
[1] 9
```

`apply()` applies a function over the rows (`MARGIN = 1`) or columns (`MARGIN = 2`) of a matrix.

```
BASE R
m <- matrix(1:6, nrow = 2)
apply(m, 1, sum)                        # row sums
apply(m, 2, max)                        # column maxima
```

```
[1]  9 12
[1] 2 4 6
```

Basic error handling

Signal problems from your own functions with `stop()` (an error that halts execution) and `warning()` (a message that lets execution continue). Catch them with `tryCatch()`, which takes the code to try and one handler per condition class; `finally` runs in every case.

```
BASE R
safe_log <- function(x) {
  if (!is.numeric(x)) stop("x must be numeric")
  if (any(x < 0, na.rm = TRUE)) warning("negative values produce NaN")
  log(x)
}
tryCatch(safe_log("a"), error = function(e) conditionMessage(e))
tryCatch(safe_log(-1),
         warning = function(w) "caught a warning",
         finally = cat("done\n"))
result <- tryCatch(safe_log("a"), error = function(e) NA_real_)
result
```

```
[1] "x must be numeric"
done
[1] "caught a warning"
[1] NA
```

Best Practice. Keep functions short and single-purpose, validate arguments at the top, give them verb names such as `clean_names` or `plot_trend`, and avoid hidden dependencies on global variables.

Exercises

5.1 [Easy] Write a function `square()` that returns the square of a number, and call it on `1:5`.

5.2 [Easy] Write `celsius_to_f(c)` returning `c * 9 / 5 + 32` and convert 0 and 100.

5.3 [Medium] Use `vapply()` to compute the length of each string in `c("R", "data", "science")`.

5.4 [Medium] Write a function `cv(x)` returning the coefficient of variation (`sd / mean`) that stops with an error when `x` is not numeric.

5.5 [Hard] Use `Map()` to build the strings "a-1", "b-2", "c-3" from `c("a", "b", "c")` and `1:3`, and return them as a character vector.

6. Importing and Exporting Data

Packages required: readr, readxl, dplyr (for glimpse). All examples write only to temporary files.

Real analysis starts with reading a file. The examples here first create a small CSV in a temporary location, so no external files are needed and nothing of yours is touched.

Reading and writing CSV files

Base R reads with `read.csv()` and writes with `write.csv()`. The readr package offers `read_csv()` and `write_csv()`, which are faster, never convert text to factors, guess column types and report them. Use `row.names = FALSE` when writing with base R, otherwise an extra index column is added.

```
PACKAGE CODE: readr
tmp <- tempfile(fileext = ".csv")            # a path in the session temp folder
write.csv(mtcars[1:5, 1:4], tmp, row.names = FALSE)
cars_base <- read.csv(tmp)
cars_base
library(readr)
cars_tbl <- read_csv(tmp, show_col_types = FALSE)
class(cars_tbl)
```

```
   mpg cyl disp  hp
1 21.0   6  160 110
2 21.0   6  160 110
3 22.8   4  108  93
4 21.4   6  258 110
5 18.7   8  360 175
[1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

Common Mistake. `write.csv()` and `write_csv()` overwrite an existing file without asking. Write results to a dedicated `output/` folder and never to the path of your raw data. Treat raw data as read-only.

Version Note. `show_col_types` exists from readr 2.0.0. Without it, `read_csv()` prints a column specification message on every call.

File paths, encoding and other formats

Use forward slashes on every platform (`"data/sales.csv"`), build paths with `file.path()` and test them with `file.exists()`. On Windows a single backslash is an escape character in strings, so `"C:\data"` is an error. For text with accents or other non-ASCII characters, state the encoding explicitly; UTF-8 is the safe default.

```
BASE R  (not run)
read.csv("data/sales.csv", fileEncoding = "UTF-8")
read_csv("data/old_export.csv", locale = locale(encoding = "latin1"))
read.csv2("data/european.csv")               # semicolon separator, comma decimals
read_delim("data/table.txt", delim = "|")    # any other delimiter
```

```
BASE R
file.exists(tmp)
basename(file.path("data", "raw", "sales.csv"))
```

```
[1] TRUE
[1] "sales.csv"
```

Excel files are read with readxl. `read_excel()` takes a path, a `sheet` (name or number) and a `range`. The package ships example workbooks, which makes the call below runnable anywhere.

```
PACKAGE CODE: readxl
library(readxl)
path <- readxl_example("datasets.xlsx")
excel_sheets(path)
read_excel(path, sheet = "mtcars", n_max = 3)
```

```
[1] "iris"     "mtcars"   "chickwts" "quakes"
# A tibble: 3 × 11
    mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  21       6   160   110  3.9   2.62  16.5     0     1     4     4
2  21       6   160   110  3.9   2.88  17.0     0     1     4     4
3  22.8     4   108    93  3.85  2.32  18.6     1     1     4     1
```

For R-only storage, `saveRDS()` and `readRDS()` keep an object exactly, including types and factor levels, in a compact binary file.

```
BASE R
rds <- tempfile(fileext = ".rds")
saveRDS(cars_base, rds)
identical(readRDS(rds), cars_base)
```

```
[1] TRUE
```

Inspecting a dataset

Inspect every dataset immediately after import. `str()` shows structure and types, `summary()` gives per-column statistics, `head()` and `tail()` show rows, and `dplyr::glimpse()` is a compact, tidyverse-style `str()`.

```
PACKAGE CODE: dplyr
str(airquality)
head(airquality, 3)
summary(airquality$Ozone)
dplyr::glimpse(iris)
```

```
'data.frame':   153 obs. of  6 variables:
 $ Ozone  : int  41 36 12 18 NA 28 23 19 8 NA ...
 $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...
 $ Wind   : num  7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
 $ Temp   : int  67 72 74 62 56 66 65 59 61 69 ...
 $ Month  : int  5 5 5 5 5 5 5 5 5 5 ...
 $ Day    : int  1 2 3 4 5 6 7 8 9 10 ...
  Ozone Solar.R Wind Temp Month Day
1    41     190  7.4   67     5   1
2    36     118  8.0   72     5   2
3    12     149 12.6   74     5   3
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
   1.00   18.00   31.50   42.13   63.25  168.00      37
Rows: 150
Columns: 5
$ Sepal.Length <dbl> 5.1, 4.9, 4.7, 4.6, 5.0, 5.4, 4.6, 5.0, 4.4, 4.9, 5.4, 4.…
$ Sepal.Width  <dbl> 3.5, 3.0, 3.2, 3.1, 3.6, 3.9, 3.4, 3.4, 2.9, 3.1, 3.7, 3.…
$ Petal.Length <dbl> 1.4, 1.4, 1.3, 1.5, 1.4, 1.7, 1.4, 1.5, 1.4, 1.5, 1.5, 1.…
$ Petal.Width  <dbl> 0.2, 0.2, 0.2, 0.2, 0.2, 0.4, 0.3, 0.2, 0.2, 0.1, 0.2, 0.…
$ Species      <fct> setosa, setosa, setosa, setosa, setosa, setosa, setosa, s…
```

Best Practice. After every import check the number of rows and columns, the column types, and the count of missing values (`colSums(is.na(df))`). Silent type errors, such as numbers read as text because of one stray character, are the most common import problem.

```
BASE R
colSums(is.na(airquality))
```

```
  Ozone Solar.R    Wind    Temp   Month     Day
     37       7       0       0       0       0
```

Exercises

6.1 [Easy] Write `head(iris, 4)` to a temporary CSV file without row names and read it back with `read.csv()`.

6.2 [Easy] How many rows and columns does `airquality` have?

6.3 [Medium] Count the missing values in each column of `airquality`.

6.4 [Medium] Save `mtcars` to a temporary RDS file, read it back, and confirm it is identical.

7. Data Wrangling

Packages required: dplyr, tidyr, stringr, lubridate (all part of the tidyverse).

Wrangling means reshaping raw data into the form an analysis needs. The dplyr package provides one verb per task, and every verb takes a data frame first and returns a data frame, so verbs chain with the pipe.

The pipe

The pipe passes the result on its left as the first argument of the function on its right: `x |> f(y)` means `f(x, y)`. Read it as "then". The native pipe `|>` requires R 4.1.0 or later. The older `%>%` comes from magrittr and is loaded with dplyr; it additionally supports a `.` placeholder anywhere in the call.

Version Note. The native pipe placeholder `_` (used with a named argument, as in `x |> lm(y ~ x, data = _)`) requires R 4.2.0. Anonymous function shorthand `\(x)` requires R 4.1.0.

Core dplyr verbs

`select()` picks columns, `filter()` picks rows, `mutate()` adds or changes columns, `arrange()` sorts, `group_by()` plus `summarise()` aggregates by group, and `count()` is a shortcut for counting rows per group.

```
PACKAGE CODE: dplyr, tibble
library(dplyr)
cars <- mtcars |> tibble::rownames_to_column("model") |> as_tibble()
cars |>
  select(model, mpg, cyl, hp) |>
  filter(cyl == 4, mpg > 30) |>
  arrange(desc(mpg))
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

# A tibble: 4 × 4
  model            mpg   cyl    hp
  <chr>          <dbl> <dbl> <dbl>
1 Toyota Corolla  33.9     4    65
2 Fiat 128        32.4     4    66
3 Honda Civic     30.4     4    52
4 Lotus Europa    30.4     4   113
```

```
BASE R
cars |>
  mutate(kpl = round(mpg * 0.425144, 1)) |>      # new column
  group_by(cyl) |>
  summarise(n = n(), mean_mpg = mean(mpg), max_hp = max(hp))
cars |> count(gear, sort = TRUE)
```

```
# A tibble: 3 × 4
    cyl     n mean_mpg max_hp
  <dbl> <int>    <dbl>  <dbl>
1     4    11     26.7    113
2     6     7     19.7    175
3     8    14     15.1    335
# A tibble: 3 × 2
   gear     n
  <dbl> <int>
1     3    15
2     4    12
3     5     5
```

Select by pattern or type with helpers such as `starts_with()` and `where()`, and apply one function to many columns with `across()`.

```
BASE R
cars |> select(model, starts_with("d")) |> head(3)
cars |>
  group_by(am) |>
  summarise(across(c(mpg, wt), \(x) round(mean(x), 2)))
```

```
# A tibble: 3 × 3
  model          disp  drat
  <chr>         <dbl> <dbl>
1 Mazda RX4       160  3.9
2 Mazda RX4 Wag   160  3.9
3 Datsun 710      108  3.85
# A tibble: 2 × 3
     am   mpg    wt
  <dbl> <dbl> <dbl>
1     0  17.2  3.77
2     1  24.4  2.41
```

Version Note. `summarise(.by = cyl)` and `mutate(.by = cyl)` (per-operation grouping) require dplyr 1.1.0 or later; `group_by()` works in all versions. `reframe()` and `join_by()` are also new in 1.1.0.

Joins

A join combines two tables on key columns. `inner_join()` keeps only matching rows, `left_join()` keeps all rows of the first table, `full_join()` keeps everything, `anti_join()` returns rows of the first table without a match.

```
BASE R
people <- tibble(id = c(1, 2, 3), name = c("Ana", "Ben", "Cy"))
scores <- tibble(id = c(1, 2, 4), score = c(90, 75, 60))
inner_join(people, scores, by = "id")
left_join(people, scores, by = "id")           # Cy has no score: NA
anti_join(people, scores, by = "id")
```

```
# A tibble: 2 × 3
     id name  score
  <dbl> <chr> <dbl>
1     1 Ana      90
2     2 Ben      75
# A tibble: 3 × 3
     id name  score
  <dbl> <chr> <dbl>
1     1 Ana      90
2     2 Ben      75
3     3 Cy       NA
# A tibble: 1 × 2
     id name
  <dbl> <chr>
1     3 Cy
```

Common Mistake. Joining on a key that is not unique silently multiplies rows. After a join, compare the row count with what you expected, and check keys with `count(key) |> filter(n > 1)`.

Reshaping with tidyr

Tidy data has one variable per column and one observation per row. `pivot_longer()` turns several columns into name-value pairs; `pivot_wider()` does the reverse.

```
PACKAGE CODE: tidyr
library(tidyr)
wide <- tibble(id = 1:2, q1 = c(5, 7), q2 = c(6, 8))
long <- pivot_longer(wide, cols = q1:q2, names_to = "quarter", values_to = "sales")
long
pivot_wider(long, names_from = quarter, values_from = sales)
```

```
# A tibble: 4 × 3
     id quarter sales
  <int> <chr>   <dbl>
1     1 q1          5
2     1 q2          6
3     2 q1          7
4     2 q2          8
# A tibble: 2 × 3
     id    q1    q2
  <int> <dbl> <dbl>
1     1     5     6
2     2     7     8
```

Missing values

Decide deliberately how to treat `NA`: remove it, replace it, or keep it and make functions skip it.

```
BASE R
aq <- as_tibble(airquality)
sum(is.na(aq$Ozone))
aq |> drop_na(Ozone, Solar.R) |> nrow()          # rows with complete data
aq |> summarise(mean_ozone = mean(Ozone, na.rm = TRUE))
aq |> mutate(Ozone = replace_na(Ozone, 0)) |> pull(Ozone) |> head(6)
coalesce(c(1, NA, 3), 0)                          # first non-missing value
```

```
[1] 37
[1] 111
# A tibble: 1 × 1
  mean_ozone
       <dbl>
1       42.1
[1] 41 36 12 18  0 28
[1] 1 0 3
```

Best Practice. Never replace missing values with 0 just to make code run. Zero is a real measurement. Use `na.rm = TRUE`, `drop_na()`, or a documented imputation.

Strings with stringr

stringr functions all start with `str_` and take the string first. Patterns are regular expressions; use `fixed()` for literal text.

```
PACKAGE CODE: stringr
library(stringr)
s <- c("  Alpha-01 ", "beta-02", "Gamma-03")
str_trim(s)
str_to_lower(str_trim(s))
str_detect(str_trim(s), "^[A-Z]")      # starts with an uppercase letter
str_replace(s, "-", "_")
str_sub(str_trim(s), 1, 3)
str_split(str_trim(s), "-", simplify = TRUE)
str_c("id_", 1:3)
str_length(s)
```

```
[1] "Alpha-01" "beta-02"  "Gamma-03"
[1] "alpha-01" "beta-02"  "gamma-03"
[1]  TRUE FALSE  TRUE
[1] "  Alpha_01 " "beta_02"     "Gamma_03"
[1] "Alp" "bet" "Gam"
     [,1]    [,2]
[1,] "Alpha" "01"
[2,] "beta"  "02"
[3,] "Gamma" "03"
[1] "id_1" "id_2" "id_3"
[1] 11  7  8
```

Dates with lubridate

lubridate parses dates from the order of their parts: `ymd()`, `dmy()`, `mdy()`. Extract components with `year()`, `month()`, `day()`, `wday()`, and round with `floor_date()`.

```
BASE R
library(lubridate, warn.conflicts = FALSE)
d <- ymd(c("2024-03-15", "2024-12-31"))
year(d); month(d); day(d)
d + days(10)
d[1] + months(1)
floor_date(d, "month")
as.numeric(d[2] - d[1])
dmy("05/03/2024")                                 # day first
```

```
[1] 2024 2024
[1]  3 12
[1] 15 31
[1] "2024-03-25" "2025-01-10"
[1] "2024-04-15"
[1] "2024-03-01" "2024-12-01"
[1] 291
[1] "2024-03-05"
```

Exercises

7.1 [Easy] From `mtcars`, keep cars with 8 cylinders and show their mean `mpg`.

7.2 [Easy] Convert the text "2024-07-04" to a date and extract the month number with lubridate.

7.3 [Medium] For `iris`, compute the mean `Sepal.Length` per `Species`.

7.4 [Medium] In `airquality`, how many rows remain after dropping rows with a missing `Ozone`?

7.5 [Hard] Reshape `tibble(id = 1:2, a = c(1, 2), b = c(3, 4))` to long format with columns `id`, `var`, `val`.

8. Data Visualization

Packages required: base R graphics, then ggplot2.

A plot is the fastest way to find errors and patterns in data. Base R draws plots in one call and is ideal for quick checks. ggplot2 builds a plot from layers following a consistent grammar and is the standard for polished figures.

Base R plots

`plot()` draws a scatter plot for two numeric vectors, `hist()` a histogram, `boxplot()` a box plot by group, and `barplot()` a bar chart of counts. Common arguments are `main`, `xlab`, `ylab`, `col`, `pch` (point shape) and `cex` (size). `par(mfrow = c(1, 2))` puts two plots side by side.

```
BASE R
plot(mtcars$wt, mtcars$mpg, pch = 19, col = "steelblue",
     main = "Fuel economy by weight",
     xlab = "Weight (1000 lb)", ylab = "Miles per gallon")
```

```
BASE R
boxplot(mpg ~ cyl, data = mtcars, col = "grey85",
        main = "Mileage by cylinders", xlab = "Cylinders", ylab = "Miles per gallon")
```

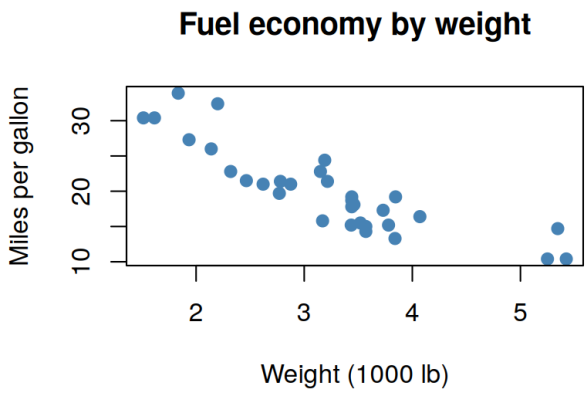


Figure 1. Base R scatter plot.

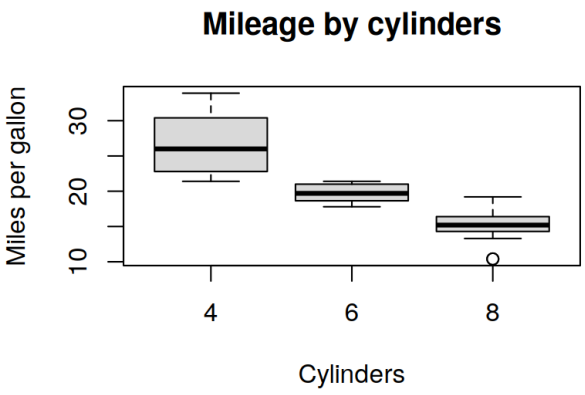


Figure 2. Base R box plot by group.

The grammar of graphics in ggplot2

Every ggplot2 plot has three required parts: the **data**, the **aesthetics** (`aes()`, which map columns to x, y, colour, size, shape), and at least one **geom** (the visual form). Layers are added with `+`. Mapping a column to an aesthetic goes inside `aes()`; setting a fixed value, such as `colour = "red"`, goes outside it.

```
PACKAGE CODE: ggplot2
library(ggplot2)
ggplot(mtcars, aes(x = wt, y = mpg, colour = factor(cyl))) +
  geom_point(size = 2) +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x) +
  labs(title = "Fuel economy by weight", x = "Weight (1000 lb)",
       y = "Miles per gallon", colour = "Cylinders") +
  theme_minimal()
```

```
BASE R
ggplot(iris, aes(x = Sepal.Length)) +
  geom_histogram(bins = 15, fill = "steelblue", colour = "white") +
  facet_wrap(~ Species) +
  labs(x = "Sepal length (cm)", y = "Count") +
  theme_minimal()
```

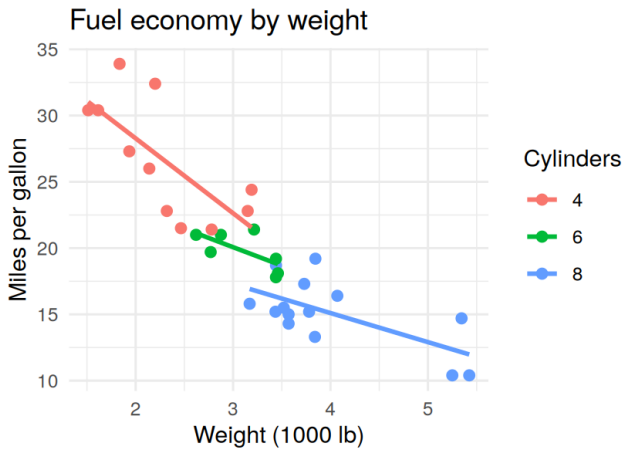


Figure 3. Points, a fitted line, labels and a theme.

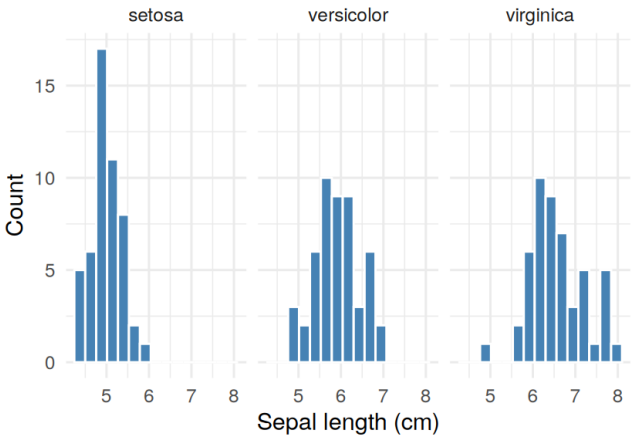


Figure 4. A histogram split into panels with facet_wrap().

The most useful chart types

Choose the geom from the question you ask of the data.

Question	Geom	Typical call
Relationship of two numbers	`geom_point()`	`aes(x = wt, y = mpg)`
Trend over time	`geom_line()`	`aes(x = date, y = value)`
Distribution of one number	`geom_histogram()`, `geom_density()`	`aes(x = value)`
Compare a number across groups	`geom_boxplot()`	`aes(x = group, y = value)`
Count categories	`geom_bar()`	`aes(x = category)`
Show precomputed bar heights	`geom_col()`	`aes(x = category, y = total)`

```
BASE R
ggplot(economics, aes(x = date, y = unemploy)) +
  geom_line(colour = "darkred") +
  labs(x = NULL, y = "Unemployed (thousands)") +
  theme_minimal()
```

```
BASE R
ggplot(mtcars, aes(x = factor(cyl), fill = factor(am))) +
  geom_bar(position = "dodge") +
  labs(x = "Cylinders", y = "Cars", fill = "Manual (1)") +
  theme_minimal()
```

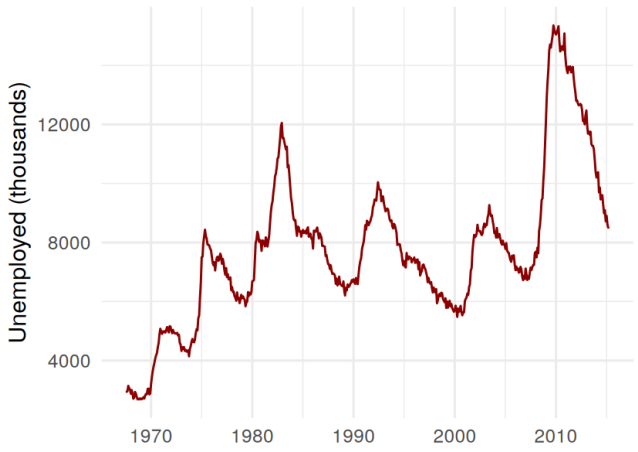


Figure 5. A time series with geom_line().

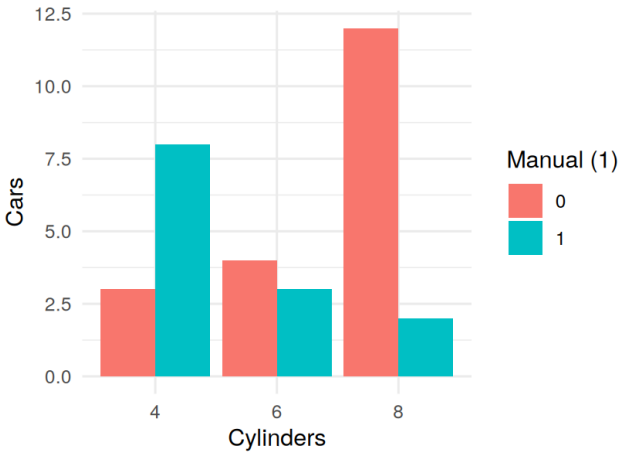


Figure 6. Counts with geom_bar() and a fill aesthetic.

Facets (`facet_wrap(~ var)`, `facet_grid(rows ~ cols)`) repeat a plot per group. Themes change the overall look (`theme_minimal()`, `theme_bw()`, `theme_classic()`), and `labs()` sets titles and axis labels. Always label axes with units.

> **Common Mistake.** Putting a fixed colour inside `aes()`: `aes(colour = "blue")` does not make points blue; it creates a legend entry named "blue" with the default palette. Write `geom_point(colour = "blue")`.

Saving plots

`ggsave()` writes the last plot, or the plot you pass, to a file; the format follows the extension (`.png`, `.pdf`, `.svg`). Set size in inches and use `dpi = 300` for print. For base graphics, open a device with `png()` or `pdf()`, draw, then close it with `dev.off()`.

```
BASE R
p <- ggplot(mtcars, aes(wt, mpg)) + geom_point()
out <- tempfile(fileext = ".png")          # a temporary file, so nothing is overwritten
ggsave(out, plot = p, width = 6, height = 4, dpi = 300)
file.exists(out)
```

```
[1] TRUE
```

> **Best Practice.** Save figures from a script with an explicit width, height and resolution, and write them to a dedicated folder. Never save a figure by screenshot: the script should regenerate every figure.

> **Version Note.** The `linewidth` argument replaced `size` for lines in ggplot2 3.4.0. In older versions use `size`. Examples here were run with ggplot2 3.4.4.

Exercises

8.1 [Easy] Create the histogram data for `mtcars$mpg` with 5 breaks without drawing it, and show the counts.

8.2 [Easy] Build a ggplot of `iris` with `Sepal.Length` on x and `Sepal.Width` on y using points, and confirm the object is a ggplot.

8.3 [Medium] Extend that plot so colour shows `Species` and add the title "Iris sepals".

8.4 [Medium] Save a plot of `airquality` (`Temp` on x, `Ozone` on y) as a PNG in a temporary folder and verify the file exists.

8.5 [Hard] Plot `ToothGrowth` as box plots of `len` by `dose` (as a factor), coloured by `supp`, and list the layer geoms used.

9. Statistics Essentials

Packages required: base R only (the stats package is attached by default).

R was built for statistics, and the core tools need no packages. This chapter covers what you need to describe data, check relationships and fit a first model. Statistical output tells you about evidence, not truth: always report effect sizes and look at a plot next to the test.

Descriptive statistics

Location is described by `mean()` and `median()`, spread by `sd()`, `var()`, `range()` and `IQR()`, and shape by `quantile()`. `summary()` gives the five-number summary plus the mean. For categories use `table()` for counts and `prop.table()` for proportions. Add `na.rm = TRUE` when the data contain missing values.

```
BASE R
x <- mtcars$mpg
mean(x); median(x); sd(x)
range(x); IQR(x)
quantile(x, c(0.25, 0.75))
summary(x)
prop.table(table(mtcars$cyl))
```

```
[1] 20.09062
[1] 19.2
[1] 6.026948
[1] 10.4 33.9
[1] 7.375
   25%    75%
15.425 22.800
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  10.40   15.43   19.20   20.09   22.80   33.90

      4       6       8
0.34375 0.21875 0.43750
```

The mean (20.09) is close to the median (19.2), so the distribution has no strong skew. The standard deviation of 6.03 miles per gallon tells you a typical car lies about that far from the mean.

Distributions and random numbers

Each distribution has four functions with a one-letter prefix: `d` density, `p` cumulative probability, `q` quantile, `r` random draws. For the normal distribution these are `dnorm()`, `pnorm()`, `qnorm()` and `rnorm()`. Call `set.seed()` first to make random results reproducible.

```
BASE R
set.seed(42)
round(rnorm(5, mean = 100, sd = 15), 1)
pnorm(1.96)                  # P(Z <= 1.96)
qnorm(0.975)                 # the value with 97.5% of the area below it
sample(1:10, 3)              # random sample without replacement
```

```
[1] 120.6  91.5 105.4 109.5 106.1
[1] 0.9750021
[1] 1.959964
[1] 8 7 4
```

> **Version Note.** `sample()` changed its algorithm in R 3.6.0 (`sample.kind = "Rejection"`), so seeded results of `sample()` differ between R 3.5 and later versions. Seeded `rnorm()` results are stable across R 4.x.

Correlation

`cor()` returns the Pearson correlation, from -1 to 1. `cor.test()` adds a confidence interval and a test of the null hypothesis that the true correlation is zero.

```
BASE R
cor(mtcars$mpg, mtcars$wt)
cor.test(mtcars$mpg, mtcars$wt)
```

```
[1] -0.8676594

        Pearson's product-moment correlation

data:  mtcars$mpg and mtcars$wt
t = -9.559, df = 30, p-value = 1.294e-10
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.9338264 -0.7440872
sample estimates:
       cor
-0.8676594
```

The correlation of -0.87 is strong and negative: heavier cars have lower mileage. The p-value (1.294e-10) is far below 0.05, and the 95 percent interval (-0.93 to -0.74) excludes zero. Correlation does not show causation.

The t-test

`t.test()` compares means. The formula `mpg ~ am` reads "mpg by group am" (0 = automatic, 1 = manual). R runs Welch's test by default, which does not assume equal variances.

```
BASE R
t.test(mpg ~ am, data = mtcars)
```

```
        Welch Two Sample t-test

data:  mpg by am
t = -3.7671, df = 18.332, p-value = 0.001374
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 -11.280194  -3.209684
sample estimates:
mean in group 0 mean in group 1
       17.14737        24.39231
```

The group means are 17.15 and 24.39. The p-value 0.001374 is below 0.05, so the data are inconsistent with equal means; the confidence interval for the difference runs from -11.28 to -3.21 miles per gallon. For a one-sample test use `t.test(x, mu = 20)`; for paired data add `paired = TRUE`.

The chi-squared test

`chisq.test()` tests whether two categorical variables are independent, using a table of counts. Here hair colour and eye colour (summed over sex) from the built-in `HairEyeColor` data.

```
BASE R
tab <- margin.table(HairEyeColor, c(1, 2))
tab
chisq.test(tab)
```

```
       Eye
Hair    Brown Blue Hazel Green
  Black    68   20    15     5
  Brown   119   84    54    29
  Red      26   17    14    14
  Blond     7   94    10    16

        Pearson's Chi-squared test

data:  tab
X-squared = 138.29, df = 9, p-value < 2.2e-16
```

The tiny p-value means hair and eye colour are not independent. When expected counts are small (below about 5) R warns that the approximation may be incorrect; use `fisher.test()` then.

Simple linear regression

`lm()` fits a linear model. The formula `mpg ~ wt` means "model mpg as a function of wt". `summary()` is the main way to read it.

```
BASE R
fit <- lm(mpg ~ wt, data = mtcars)
summary(fit)
confint(fit)
predict(fit, newdata = data.frame(wt = 3))
```

```
Call:
lm(formula = mpg ~ wt, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.2851     1.8776  19.858  < 2e-16 ***
wt           -5.3445     0.5591  -9.559 1.29e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared:  0.7528,    Adjusted R-squared:  0.7446
F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10

                2.5 %    97.5 %
(Intercept) 33.450500 41.119753
wt          -6.486308 -4.202635
       1
21.25171
```

How to read the summary: **Estimate** is the fitted coefficient. The slope of -5.34 means each additional 1000 lb of weight lowers predicted mileage by about 5.3 miles per gallon; the intercept 37.29 is the prediction at weight zero. **Std. Error** is the uncertainty of each estimate. The **t value** is the estimate divided by its standard error, and **Pr(>|t|)** is the p-value for the null hypothesis that the coefficient is zero. **Multiple R-squared** (0.7528) is the share of variance in mileage explained by weight. The **residual standard error** (3.046) is the typical prediction error in the units of the outcome. The **F-statistic** tests the model as a whole.

Common Mistake. Treating a small p-value as proof of an important effect, and extrapolating a model beyond the range of the data. Weights in `mtcars` run from about 1.5 to 5.4 (thousand pounds); the intercept at weight 0 is a mathematical anchor, not a real car.

Best Practice. Plot the data before and after fitting a model, report effect sizes with confidence intervals (`confint()`), and write down the question before choosing the test.

Exercises

9.1 [Easy] Compute the mean and standard deviation of `iris$Petal.Length`.

9.2 [Easy] Using the standard normal distribution, find the probability of a value below 1.

9.3 [Medium] Compute the correlation between `Ozone` and `Temp` in `airquality`, ignoring missing rows.

9.4 [Medium] Test whether mean `len` differs between the two `supp` groups of `ToothGrowth` and extract the p-value.

9.5 [Hard] Fit `mpg ~ hp` on `mtcars` and report the slope and R-squared.

10. Mini Projects

Packages required: dplyr, tidyr, ggplot2, lubridate. All data are built in; nothing is read from or written to your folders.

Each project is complete and runs from a fresh session. Read the comments, run it, then change something.

Project 1: clean and summarize a dataset

Goal: turn the `airquality` data (daily New York air measurements, May to September 1973) into a clean table and a monthly summary. The steps are the standard cleaning sequence: inspect, fix types, rename, handle missing values, derive, summarize.

```
BASE R
library(dplyr, warn.conflicts = FALSE)       # quiet: no masking report
library(lubridate, warn.conflicts = FALSE)

# 1. Inspect the raw data
raw <- as_tibble(airquality)
colSums(is.na(raw))

# 2. Clean: build a date, rename columns, convert units
clean <- raw |>
  mutate(date = make_date(1973, Month, Day),          # year was not stored
         temp_c = round((Temp - 32) * 5 / 9, 1)) |>   # Fahrenheit to Celsius
  rename(ozone = Ozone, solar = Solar.R, wind = Wind) |>
  select(date, ozone, solar, wind, temp_c)

# 3. Summarize by month; keep missing values out of the averages
monthly <- clean |>
  group_by(month = month(date)) |>
  summarise(days = n(),
            missing_ozone = sum(is.na(ozone)),
            mean_ozone = round(mean(ozone, na.rm = TRUE), 1),
            mean_temp_c = round(mean(temp_c), 1))
monthly

# 4. Check a result against the raw data
clean |> filter(!is.na(ozone)) |> slice_max(ozone, n = 1)
```

```
  Ozone Solar.R    Wind    Temp   Month     Day
     37       7       0       0       0       0
# A tibble: 5 × 5
  month  days missing_ozone mean_ozone mean_temp_c
  <dbl> <int>         <int>      <dbl>       <dbl>
1     5    31             5       23.6        18.6
2     6    30            21       29.4        26.2
3     7    31             5       59.1        28.8
4     8    31             5       60          28.9
5     9    30             1       31.4        25
# A tibble: 1 × 5
  date       ozone solar  wind temp_c
  <date>     <int> <int> <dbl>  <dbl>
1 1973-08-25   168   238   3.4   27.2
```

Project 2: publication-quality plots

Goal: three consistent figures from one dataset. A shared theme and palette keep a set of plots looking like one document. Colours are given as hex codes so the result does not depend on extra packages.

```
PACKAGE CODE: ggplot2
library(ggplot2)
aq <- transform(airquality, Month = factor(Month, labels = month.abb[5:9]))
base_theme <- theme_minimal(base_size = 11) +
  theme(plot.title = element_text(face = "bold"), legend.position = "bottom")
ggplot(aq, aes(Month, Temp)) +
  geom_boxplot(fill = "#9ecae1", colour = "#08519c") +
  labs(title = "Temperature by month", x = NULL, y = "Temperature (F)") +
  base_theme
```

```
BASE R
ggplot(aq, aes(Temp, Ozone)) +
  geom_point(alpha = 0.7, colour = "#08519c") +
  geom_smooth(method = "loess", formula = y ~ x, colour = "#cb181d", se = TRUE) +
  labs(title = "Ozone rises with temperature", x = "Temperature (F)",
       y = "Ozone (ppb)") +
  base_theme
```

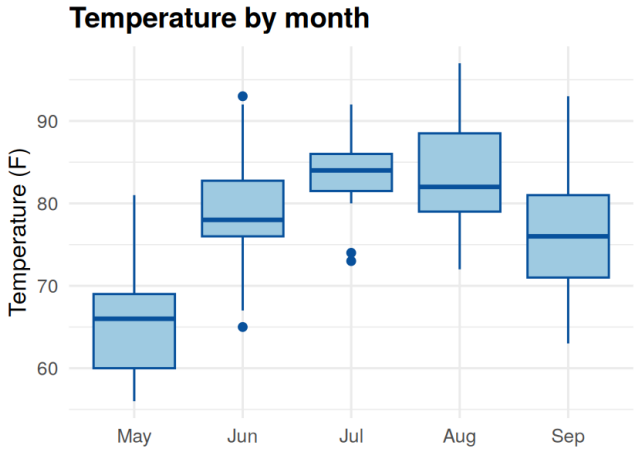


Figure 7. Temperature distribution by month.

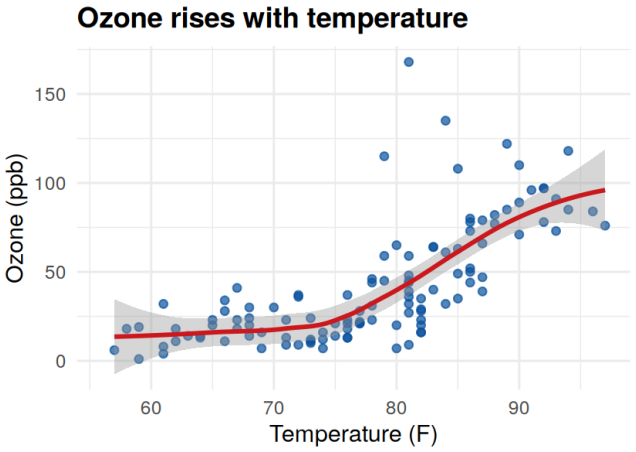


Figure 8. Ozone against temperature with a smoother.

```
BASE R
se <- function(v) sd(v) / sqrt(length(v))              # standard error of the mean
summ <- data.frame(Month = levels(aq$Month),
                   m = tapply(aq$Ozone, aq$Month, mean, na.rm = TRUE),
                   se = tapply(aq$Ozone, aq$Month, \(v) se(na.omit(v))))
summ$Month <- factor(summ$Month, levels = month.abb[5:9])
ggplot(summ, aes(Month, m)) +
  geom_col(fill = "#6baed6") +
  geom_errorbar(aes(ymin = m - se, ymax = m + se), width = 0.25) +
  labs(title = "Mean ozone by month", x = NULL, y = "Ozone (ppb)") +
  base_theme
```

```
BASE R
ggplot(aq, aes(Wind, Ozone)) +
  geom_point(colour = "#238b45", alpha = 0.8) +
  facet_wrap(~ Month, nrow = 2) +
  labs(title = "Windier days, lower ozone", x = "Wind (mph)", y = "Ozone (ppb)") +
  base_theme
```

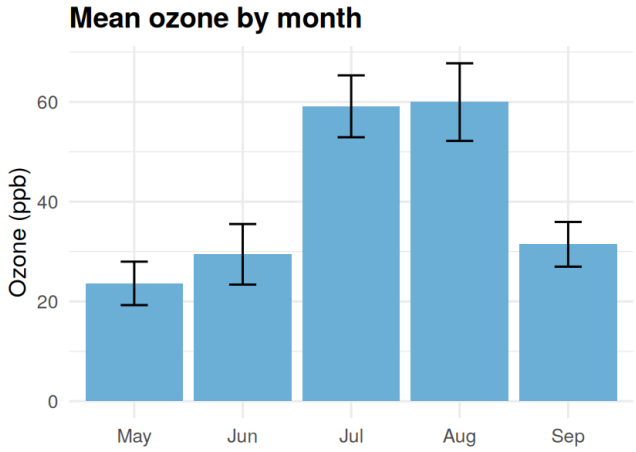


Figure 9. Mean ozone by month with error bars.

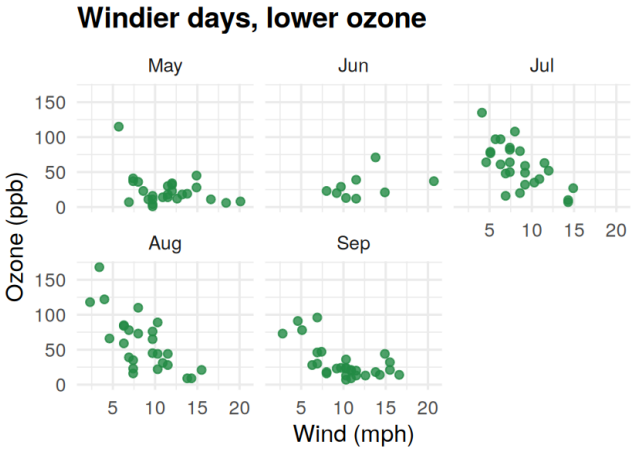


Figure 10. Wind and ozone, faceted by month.

Project 3: a regression analysis with interpretation

Goal: explain fuel economy in `mtcars` using weight and horsepower, check the model and make a prediction.

```
BASE R
# 1. Explore the relationships first
round(cor(mtcars[, c("mpg", "wt", "hp")]), 2)

# 2. Fit the model
fit <- lm(mpg ~ wt + hp, data = mtcars)
summary(fit)
```

```
      mpg    wt    hp
mpg  1.00 -0.87 -0.78
wt  -0.87  1.00  0.66
hp  -0.78  0.66  1.00

Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:
   Min     1Q Median     3Q    Max
-3.941 -1.600 -0.182  1.050  5.854

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.22727    1.59879  23.285  < 2e-16 ***
wt          -3.87783    0.63273  -6.129 1.12e-06 ***
hp          -0.03177    0.00903  -3.519  0.00145 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.593 on 29 degrees of freedom
Multiple R-squared:  0.8268,    Adjusted R-squared:  0.8148
F-statistic: 69.21 on 2 and 29 DF,  p-value: 9.109e-12
```

```
BASE R
# 3. Uncertainty and prediction
round(confint(fit), 2)
new_car <- data.frame(wt = 3, hp = 120)
predict(fit, new_car, interval = "prediction")

# 4. Compare with the simpler model
round(c(wt_only = summary(lm(mpg ~ wt, mtcars))$adj.r.squared,
        wt_hp = summary(fit)$adj.r.squared), 3)
```

```
            2.5 % 97.5 %
(Intercept) 33.96  40.50
wt          -5.17  -2.58
hp          -0.05  -0.01
       fit      lwr      upr
1 21.78102 16.38174 27.18031
wt_only   wt_hp
  0.745   0.815
```

```
BASE R
plot(fit, which = 1, id.n = 0)
```

```
BASE R
plot(fit, which = 2, id.n = 0)
```

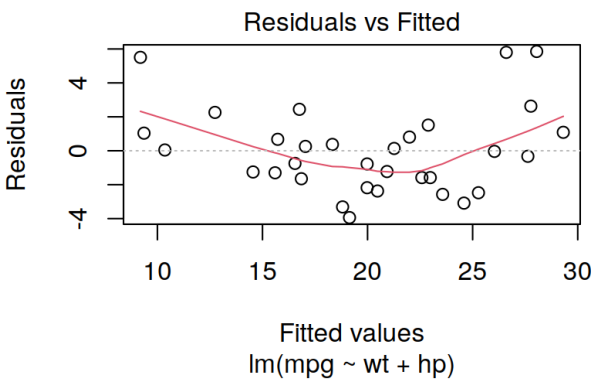


Figure 11. Residuals against fitted values.

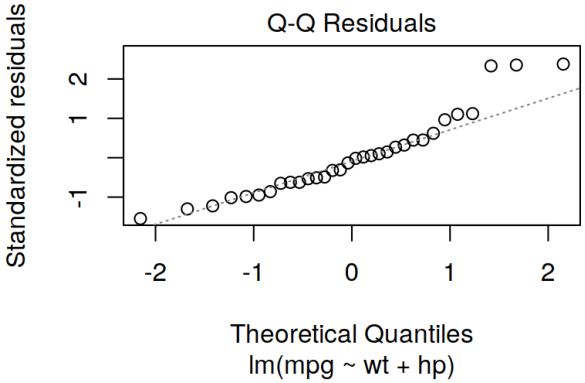


Figure 12. Normal Q-Q plot of the residuals.

Interpretation. Holding horsepower fixed, each extra 1000 lb of weight is associated with 3.88 fewer miles per gallon; holding weight fixed, each extra horsepower lowers mileage by about 0.03. Both coefficients are significant at the 0.01 level, and the model explains 82 percent of the variance (R-squared 0.8268, adjusted 0.8148), noticeably more than weight alone (adjusted 0.745). The residual plot should show no strong pattern and the Q-Q plot should follow the line; deviations at the ends point to a few unusual cars rather than a broken model. The prediction interval is wide because it describes a single new car, not an average. Remember that the data are 32 cars from 1974: the model describes association in this sample, not a law of nature.

Exercises

10.1 [Easy] In Project 1, which month has the highest mean ozone? Compute it directly from `airquality`.

10.2 [Medium] Add `qsec` to the Project 3 model and compare adjusted R-squared with the two-predictor model.

10.3 [Hard] Fit `mpg ~ wt + factor(am)` and report the estimated difference between manual and automatic cars at equal weight.

Appendix: Cheat Sheet and Next Steps

Most used functions

Task	Functions
Inspect	`str()`, `summary()`, `head()`, `tail()`, `dim()`, `names()`, `class()`, `dplyr::glimpse()`
Create vectors	`c()`, `:`, `seq()`, `rep()`, `numeric(n)`, `vector("list", n)`
Subset	`x[i]`, `x[[i]]`, `x$name`, `df[rows, cols]`, `which()`, `%in%`, `head()`
Summarize	`mean()`, `median()`, `sd()`, `quantile()`, `table()`, `cumsum()`, `colSums()`, `rowSums()`
Missing data	`is.na()`, `na.rm = TRUE`, `complete.cases()`, `tidyr::drop_na()`, `tidyr::replace_na()`
Text	`paste()`, `paste0()`, `nchar()`, `toupper()`, `stringr::str_detect()`, `str_replace()`, `str_sub()`
Dates	`as.Date()`, `format()`, `lubridate::ymd()`, `year()`, `month()`, `floor_date()`
Wrangle	`select()`, `filter()`, `mutate()`, `arrange()`, `group_by()`, `summarise()`, `count()`, `left_join()`
Reshape	`pivot_longer()`, `pivot_wider()`, `rbind()`, `cbind()`
Iterate	`for`, `while`, `ifelse()`, `lapply()`, `sapply()`, `vapply()`, `Map()`, `apply()`
Functions	`function(x) {}`, `\(x) x + 1`, `stop()`, `warning()`, `tryCatch()`, `stopifnot()`
Import/export	`read.csv()`, `readr::read_csv()`, `readxl::read_excel()`, `write.csv()`, `saveRDS()`, `readRDS()`
Plot	`plot()`, `hist()`, `boxplot()`, `ggplot()`, `geom_point()`, `geom_line()`, `facet_wrap()`, `ggsave()`
Statistics	`cor()`, `t.test()`, `chisq.test()`, `lm()`, `summary()`, `confint()`, `predict()`, `set.seed()`

Task	Functions
Help	`?name`, `??topic`, `args()`, `example()`, `vignette()`

Common errors and fixes

Message	Usual cause and fix
`could not find function "x"`	Package not loaded (`library()`) or the function name is misspelled
`object 'x' not found`	Typo, object not yet created, or a column name used outside `aes()`/`dplyr` verbs
`unexpected symbol` / `unexpected '}'`	Missing comma, operator or bracket; check the line above the reported one
`non-numeric argument to binary operator`	Arithmetic on text; convert with `as.numeric()` and inspect with `str()`
`subscript out of bounds`	Index beyond the length or dimension; remember that counting starts at 1
`arguments imply differing number of rows`	Columns of unequal length in `data.frame()`
`the condition has length > 1`	Vector used in `if`; use `any()`, `all()` or `ifelse()`
`NAs introduced by coercion`	Text that is not a number was converted; find it with `is.na(as.numeric(x))`
`cannot open file '...'`	Wrong path or working directory; check `getwd()` and `file.exists()`
`there is no package called 'x'`	Run `install.packages("x")` once, then `library(x)`

Next steps and resources

Practice on your own data: choose a question, import the data, clean it, plot it, model it and write down the conclusion. Then widen your skills in this order.

Resource	Use
R for Data Science, 2nd edition (r4ds.hadley.nz)	Free book; the natural continuation of Chapters 6 to 8
Advanced R (adv-r.hadley.nz)	Free book on how the language works, once functions feel comfortable
Tidyverse style guide (style.tidyverse.org)	The coding style used in this guide
Package websites (dplyr.tidyverse.org, ggplot2.tidyverse.org)	Reference pages, cheat sheets and articles for each package
Posit cheat sheets (posit.co/resources/cheatsheets)	One-page summaries of core packages
CRAN (cran.r-project.org)	Official R downloads, package list and manuals

Topics to learn next: R Markdown or Quarto for reproducible reports, `purrr` for iteration, `broom` for tidy model output, generalized linear models (`glm()`), and Git for version control. Ask questions with a minimal reproducible example: the smallest code and data that show the problem.

Answers to Exercises

Each answer is self-contained and was run in a fresh R session. Lines beginning with #> show the result, where short.

Chapter 1: Setup and First Steps

1.1 (Easy)

```
(15 + 25) / 4
47 %% 6
#> [1] 10
#> [1] 5
```

1.2 (Easy)

```
args(round)
#> function (x, digits = 0)
#> NULL
```

1.3 (Medium)

```
x <- (1:10)^2
mean(x)
#> [1] 38.5
```

1.4 (Medium)

```
a <- 1
ls()
rm(list = ls())
ls()
#> [1] "a"
#> character(0)
```

Chapter 2: Data Types and Variables

2.1 (Easy)

```
class(7L)
class(7)
#> [1] "integer"
#> [1] "numeric"
```

2.2 (Easy)

```
isTRUE(all.equal(0.1 + 0.2, 0.3))
#> [1] TRUE
```

2.3 (Medium)

```
v <- suppressWarnings(as.numeric(c("10", "20", "x")))
sum(is.na(v))
#> [1] 1
```

2.4 (Medium)

```
f <- factor(c("S", "L", "M", "S", "L"), levels = c("S", "M", "L"), ordered = TRUE)
table(f)
#> f
#> S M L
#> 2 1 2
```

2.5 (Hard)

```
start <- as.Date("2024-01-01")
as.numeric(as.Date("2024-12-25") - start)
start + 100
#> [1] 359
#> [1] "2024-04-10"
```

Chapter 3: Data Structures

3.1 (Easy)

```
x <- c(4, 8, 15, 16, 23, 42)
x[length(x)]
x[x > 10]
#> [1] 42
#> [1] 15 16 23 42
```

3.2 (Easy)

```
p <- list(name = "Sam", age = 30, hobbies = c("chess", "rowing", "baking"))
p$hobbies[2]
#> [1] "rowing"
```

3.3 (Medium)

```
m <- matrix(1:9, nrow = 3, byrow = TRUE)
rowSums(m)
#> [1]  6 15 24
```

3.4 (Medium)

```
mtcars[mtcars$cyl == 6, c("mpg", "hp")]
```

3.5 (Hard)

```
d <- mtcars[1:10, c("mpg", "cyl")]
d$kpl <- d$mpg * 0.425144
head(d[order(-d$kpl), ], 3)
```

Chapter 4: Control Flow

4.1 (Easy)

```
n <- 17
if (n %% 2 == 0) print("even") else print("odd")
#> [1] "odd"
```

4.2 (Easy)

```
ifelse(c(5, 12, 20, 3) > 10, "high", "low")
#> [1] "low"  "high" "high" "low"
```

4.3 (Medium)

```
f <- 1
for (i in 1:10) f <- f * i
c(f, factorial(10))
#> [1] 3628800 3628800
```

4.4 (Medium)

```
p <- 1
while (p <= 1000) p <- p * 3
p
#> [1] 2187
```

4.5 (Hard)

```
n <- 1:100
sum(n %% 3 == 0 | n %% 5 == 0)
#> [1] 47
```

Chapter 5: Functions

5.1 (Easy)

```
square <- function(x) x^2
square(1:5)
#> [1]  1  4  9 16 25
```

5.2 (Easy)

```
celsius_to_f <- function(c) c * 9 / 5 + 32
celsius_to_f(c(0, 100))
#> [1]  32 212
```

5.3 (Medium)

```
vapply(c("R", "data", "science"), nchar, integer(1))
#>       R    data science
#>       1       4       7
```

5.4 (Medium)

```
cv <- function(x) {
  if (!is.numeric(x)) stop("x must be numeric")
  sd(x) / mean(x)
}
round(cv(c(10, 12, 14)), 3)
#> [1] 0.167
```

5.5 (Hard)

```
unlist(Map(function(l, i) paste(l, i, sep = "-"), c("a", "b", "c"), 1:3), use.names = FALSE)
#> [1] "a-1" "b-2" "c-3"
```

Chapter 6: Importing and Exporting Data

6.1 (Easy)

```
f <- tempfile(fileext = ".csv")
write.csv(head(iris, 4), f, row.names = FALSE)
read.csv(f)
```

6.2 (Easy)

```
dim(airquality)
#> [1] 153   6
```

6.3 (Medium)

```
colSums(is.na(airquality))
#>   Ozone Solar.R    Wind    Temp   Month     Day
#>      37       7       0       0       0       0
```

6.4 (Medium)

```
f <- tempfile(fileext = ".rds")
saveRDS(mtcars, f)
identical(readRDS(f), mtcars)
#> [1] TRUE
```

Chapter 7: Data Wrangling

7.1 (Easy)

```
library(dplyr)
mtcars |> filter(cyl == 8) |> summarise(mean_mpg = mean(mpg))
```

7.2 (Easy)

```
library(lubridate)
month(ymd("2024-07-04"))
```

7.3 (Medium)

```
library(dplyr)
iris |> group_by(Species) |> summarise(mean_len = mean(Sepal.Length))
```

7.4 (Medium)

```
library(dplyr)
library(tidyr)
airquality |> drop_na(Ozone) |> nrow()
```

7.5 (Hard)

```
library(tidyr)
library(tibble)
tibble(id = 1:2, a = c(1, 2), b = c(3, 4)) |>
  pivot_longer(c(a, b), names_to = "var", values_to = "val")
```

Chapter 8: Data Visualization

8.1 (Easy)

```
h <- hist(mtcars$mpg, breaks = 5, plot = FALSE)
h$counts
#> [1]  6 12  8  2  4
```

8.2 (Easy)

```
library(ggplot2)
p <- ggplot(iris, aes(Sepal.Length, Sepal.Width)) + geom_point()
inherits(p, "ggplot")
#> [1] TRUE
```

8.3 (Medium)

```
library(ggplot2)
p <- ggplot(iris, aes(Sepal.Length, Sepal.Width, colour = Species)) +
  geom_point() + labs(title = "Iris sepals")
p$labels$title
#> [1] "Iris sepals"
```

8.4 (Medium)

```
library(ggplot2)
p <- ggplot(airquality, aes(Temp, Ozone)) + geom_point()
f <- tempfile(fileext = ".png")
suppressWarnings(ggsave(f, p, width = 5, height = 4, dpi = 150))
file.exists(f)
#> [1] TRUE
```

8.5 (Hard)

```
library(ggplot2)
p <- ggplot(ToothGrowth, aes(factor(dose), len, fill = supp)) + geom_boxplot()
class(p$layers[[1]]$geom)[1]
#> [1] "GeomBoxplot"
```

Chapter 9: Statistics Essentials

9.1 (Easy)

```
c(mean = mean(iris$Petal.Length), sd = sd(iris$Petal.Length))
#>     mean       sd
#> 3.758000 1.765298
```

9.2 (Easy)

```
pnorm(1)
#> [1] 0.8413447
```

9.3 (Medium)

```
cor(airquality$Ozone, airquality$Temp, use = "complete.obs")
#> [1] 0.6983603
```

9.4 (Medium)

```
t.test(len ~ supp, data = ToothGrowth)$p.value
#> [1] 0.06063451
```

9.5 (Hard)

```
fit <- lm(mpg ~ hp, data = mtcars)
c(slope = coef(fit)[["hp"]], r2 = summary(fit)$r.squared)
#>       slope          r2
#> -0.06822828  0.60243734
```

Chapter 10: Mini Projects

10.1 (Easy)

```
aggregate(Ozone ~ Month, data = airquality, FUN = mean)
```

10.2 (Medium)

```
f2 <- lm(mpg ~ wt + hp, data = mtcars)
f3 <- lm(mpg ~ wt + hp + qsec, data = mtcars)
round(c(two = summary(f2)$adj.r.squared, three = summary(f3)$adj.r.squared), 4)
#>    two  three
#> 0.8148 0.8171
```

10.3 (Hard)

```
round(coef(lm(mpg ~ wt + factor(am), data = mtcars))[["factor(am)1"]], 2)
#> [1] -0.02
```